

Maven in Java – Complete Guide

1. What is Maven?

Maven is a powerful project management and build automation tool used primarily for Java projects. It simplifies the build process, dependency management, and project documentation through a standard way.

2. Importance of Maven (with Examples)

- Dependency Management – Maven automatically downloads libraries and manages their versions.
- Consistent Build System – Ensures that your project builds the same way every time.
- Integration with IDEs – Works seamlessly with NetBeans, Eclipse, IntelliJ, etc.
- Easy Project Setup – Quickly bootstrap projects using archetypes.

Example 1: Adding a Dependency

To use MySQL Connector in your Java project, you simply add this to your pom.xml: `mysql:mysql-connector-java 8.0.33`

Example 2: Building a Project

Run the command ``mvn clean install`` in terminal, and Maven will compile your code, run tests, and package it into a JAR file.

3. About pom.xml File

The pom.xml (Project Object Model) is the heart of a Maven project. It contains information about the project and configuration details like dependencies, plugins, build directory, etc.

Key Elements of pom.xml:

- `groupId` : Defines the group or company name.
- `artifactId` : The name of the project or module.
- `version` : Version of the project.
- `dependencies` : External libraries required.

4. Steps to Create Maven Project in Java (NetBeans)

- Open NetBeans IDE.
- Go to File > New Project > Maven > Java Application.
- Enter project name and location.
- Finish. NetBeans will generate a Maven project with default structure.

5. Project Structure of Maven

- src/main/java: Java source code - src/main/resources: Configuration files (e.g., application.properties) - src/test/java: Unit test code - target/: Output directory (e.g., compiled classes and final JARs) - pom.xml: Project Object Model file

